Fleet Studio

PROJECT BRIEF · RESEARCH, PROPOSAL & BUILD

Agentic QA Framework for Developer Self-Testing

One QA engineer builds the framework; developers test their own work with it; QA owns the hard edges

Prepared for Senior QA Engineer
Owner QA Engineering — Fleet Studio
Version Draft v0.2
Date 8 July 2026

1. The Shift We're Making

Our developers already have AI writing their code — Cursor, GitHub Copilot. Testing hasn't kept up: it's still a separate, mostly manual, downstream step, and with a single QA engineer it becomes the bottleneck the whole team waits on.

This project builds the testing counterpart to the developers' coding assistant: a systemic framework of AI agents that lives in the developer's own loop and tests their work as they build it. Developers self-serve the routine testing of their own changes, with the agents doing the labour — writing the test cases, running discovery and coverage, and driving a real browser to execute and report. All of it happens before a change ever reaches QA.

That changes what the QA engineer does. Instead of manually running the same checks on everyone's work, one lean QA engineer builds and owns this framework, and spends the freed time on the testing that needs judgement rather than repetition: edge cases, corner cases, cross-feature interactions, and exploratory and risk-based testing.

> **The leverage:** the framework scales to the whole team, so the person doesn't have to. It's a platform-team model applied to QA — one engineer builds the paved road, every developer drives on it.

2. The Operating Model

The core idea is a split of responsibilities. Developers own testing their own work; the framework does the labour; the QA engineer owns the framework and the judgement-heavy testing.

WHO DOES WHAT

Role	Owns
Developer	Runs the framework on their own change; reviews and fixes what it reports, before requesting QA
The framework (agents)	Writes test cases, runs discovery + coverage, executes in a browser, reports results
QA engineer	Builds and maintains the framework; sets the standard; audits output; owns edge, corner and exploratory testing; gatekeeps release-critical paths

HOW THE QA ENGINEER'S DAY CHANGES

Before	After
Manually writes and runs happy-path tests for every feature	Builds the agent framework once and improves it continuously
Is the bottleneck every change waits on	Developers self-serve routine testing; QA is no longer in the critical path for it
Time consumed by repetitive verification	Time spent on edge cases, corner cases, integrations and exploratory testing
Scales only as far as one person's hours	Scales with the framework across the whole team

3. What the Framework Does

Triggered at the point of change — in the IDE, from a CLI, or on a pull request — the framework runs three agent capabilities and reports straight back to the developer where they already work.

Capability 1 — Write the test cases

- **Reads the ticket / requirement and the code diff** to understand what the change is meant to do.
- **Generates test cases** — happy path plus key negative and boundary scenarios — and the runnable scripts for them, following the team's conventions.

Capability 2 — Discovery & code coverage

- **Discovery / impact analysis** — works out what changed and what downstream areas the change touches, so testing targets the right surface.
- **Coverage** — measures what the generated tests actually exercise, flags the gaps, and loops back to Capability 1 to close them.

Capability 3 — Run in a browser & report

- **Opens a real browser** and drives it through the test cases against a running build or preview environment.
- **Captures evidence** — pass/fail, screenshots, traces, console and network errors.
- **Reports back in the developer's flow** — a clear result on the pull request or in the IDE: what passed, what failed, and the likely defect.

Tying it together: an orchestrator sequences the three capabilities on each change and manages state and retries. The developer gets a self-test they run on demand; the QA engineer configures the standard the agents follow.

4. The QA Engineer Handles the Rest

Automating the routine is what frees the QA engineer to do the work agents are weakest at. Their expanded role:

- **Framework author & owner** — builds, maintains and improves the agents, prompts, templates and guardrails.
- **Standard-setter** — defines what a good test and adequate coverage look like, and the patterns generated tests must follow.
- **Auditor of the machine** — samples generated tests and coverage for quality and tunes the framework when it drifts.
- **Owner of judgement-heavy testing** — edge cases, corner cases, cross-feature and integration scenarios, exploratory and risk-based testing, and non-functional concerns.
- **Escalation & release gate** — developers self-serve the routine; anything ambiguous or release-critical routes to QA for sign-off.

5. Guardrails — Making "Developers Test Their Own Work" Safe

The obvious risk is developers marking their own homework. The framework is designed so QA owns the standard, not each individual test:

- **QA owns the standard, not every test** — the agents enforce patterns the QA engineer defines centrally.
- **Coverage & quality gates in CI** — changes can't merge below the agreed bar on changed code.

- **QA audits a sample** — not everything, but enough to catch drift and keep quality honest.
- **Release-critical paths still route through QA** — high-risk areas are never fully self-served.
- **Everything is traceable** — generated cases link back to requirements and to the change that triggered them.

6. It Fits the Workflow You Already Have

The framework plugs into the same surfaces developers already use with Cursor and Copilot — no heavy new process. Candidate trigger points, to be decided in research:

Surface	Use
IDE	Developer runs the self-test on their branch as they work
CLI	A single command runs the full pipeline locally
Pull request / CI	Runs automatically on push; posts results as a PR comment and enforces gates

Results land where developers already are, so self-testing becomes part of the normal build loop rather than a separate hand-off.

7. Work to Research, Propose & Build It

Phase 0 — Framing

- **Confirm the model, pick a pilot team/repo, and agree the success metrics.**

Deliverable: framing note + success metrics.

Phase 1 — Research

- **Evaluate the browser-execution approach, discovery/coverage tooling, orchestration, and trigger surface (§8).**

– **Prototype the riskiest step in isolation — most likely the browser run-and-report loop.**

Deliverable: research findings + recommended direction.

Phase 2 — Proposal & design

– **Design the agents, orchestration, the standard/guardrail model, and how it plugs into the dev loop.**

Deliverable: design document + architecture diagram.

Phase 3 — Build a thin slice

– **One change → cases → coverage → browser run → PR report, working end to end on the pilot repo.**

Deliverable: working prototype.

Phase 4 — Pilot with real developers

– **Put it in front of the pilot team on real changes; measure adoption and quality.**

Deliverable: pilot evaluation with metrics.

Phase 5 — Roll out & document

– **Harden guardrails, write the developer guide and the QA maintenance guide, and widen to more teams.**

Deliverable: documented framework + rollout guides.

8. Key Research Questions

Question	What to decide
Browser execution	How agents drive the browser and report — scripted (e.g. Playwright), Playwright + MCP, an agentic browser (browser-use / computer-use), or a hybrid
Discovery & coverage	Impact-analysis and coverage tooling for our stacks, and how gaps feed back to test generation
Orchestration	Framework to sequence the agents with good observability and retries
Trigger surface	IDE vs CLI vs PR/CI first — and how results reach the developer

Model & cost	Which model(s) for reasoning vs code vs browser control, and cost per run
Standards enforcement	How the QA engineer's standard is expressed once and applied to every generated test

9. Success Criteria

On the pilot team and at least one more, the framework is working if:

- **Developers self-serve the routine** — most happy-path and basic negative testing is done by developers via the framework, not by QA.
- **QA time is reallocated** — the QA engineer's hours shift measurably toward edge, corner and exploratory testing.
- **Coverage rises on changed code** and stays above the agreed gate.
- **Escaped defects fall** — fewer routine bugs reach QA or production.
- **It scales** — one QA engineer sustainably supports the whole team on this model.

10. Risks & Mitigations

Risk	Mitigation
Developers rubber-stamp their own tests	QA owns the standard, coverage gates in CI, QA audits a sample
Browser runs are flaky	Bound v1 to stable flows; capture traces; track and quarantine flakiness
Tests look right but assert the wrong thing	Traceability to requirements; QA audit; critic step before scripts are trusted
Runaway model cost	Cost limits, caching, cheaper models for non-reasoning steps, run only on changed surface
Low adoption by developers	Meet them in their existing tools; make the self-test one command; fast feedback
QA becomes a framework silo	Document the framework; keep the QA maintenance guide current

11. Deliverables & Indicative Timeline

Phase	Deliverable	Indicative effort

0	Framing note + success metrics	~1 Day
1	Research findings + direction	~2 Days
2	Design document + architecture	~1 Day
3	Working prototype (thin slice)	~5 Days
4	Pilot evaluation with metrics	~2 Days
5	Documented framework + rollout guides	~1 Day

Next step: confirm this framing, pick the pilot team and repo, agree the success metrics (Phase 0), then start the research phase — leading with the browser run-and-report loop, the riskiest piece.